JOURNAL OF ALGORITHMS 13, 211–234 (1992)

Cubesort: A Parallel Algorithm for Sorting N Data Items with S-Sorters

ROBERT CYPHER AND JORGE L. C. SANZ

IBM Almaden Research Center,
San Jose, California 95120

Received October 1990; accepted February 1991

This paper presents a parallel sorting algorithm called Cubesort. Cubesort sorts N data items by performing a number of rounds, each of which partitions the N data items into groups of size S and sorts within the groups. For many values of N and S, Cubesort requires fewer such rounds than are required by any previously published algorithm. Cubesort can also be used to sort N data items on hypercube, shuffle-exchange, and cube-connected cycles computers with P processors in time $O(N \log^2 N/P \log(N/P))$ over a wide range of the parameters N and P. In particular, when $N = P^{1+1/k}$ and k is a constant, Cubesort sorts on the above parallel computers in $O(N \log N/P)$ time, thus obtaining an optimal processor-time product for comparison sorting. The application of Cubesort to general routing problems is also discussed. © 1992 Academic Press, Inc.

1. INTRODUCTION

It is often useful to decompose a large sorting problem into a collection of smaller sorting problems. For example, consider a hardware device which can only sort S data items at a time. Such a device can be used to sort $N > S$ data items if a sorting problem of size N can be reduced to a sequence of sorting problems of size S. As another example, consider a collection of P hardware devices, each of which can sort S data items at a time. These devices can be used to sort $N = PS$ data items if a sorting problem can be reduced to a sequence of rounds, each of which consists of P independent (that is, non-overlapping) sorts of size S.

A number of researchers have developed techniques which solve a large sorting problem by performing a collection of smaller sorting problems [2, 4, 7, 18, 24, 25]. These techniques can be divided into sequential techniques, in which only one device capable of sorting S data items is

0196-6774/92 $5.00
Copyright © 1992 by Academic Press, Inc.
All rights of reproduction in any form reserved.

used, and parallel techniques, in which $P = N/S$ such devices are used. In this paper we focus on parallel techniques for performing such sorts. More formally, we will call an algorithm an (N, S) *sorting algorithm* if it sorts N data items by performing a sequence of rounds, where each round partitions the data into groups of S (or fewer) items and sorts within the groups. The performance of an (N, S) sorting algorithm is given by the number of rounds which it requires.

Scherson, Sen, and Shamir designed an $(N, N^{1/2})$ sorting algorithm that requires $O(\log N)$ rounds [24]. Subsequently, Schnorr and Shamir created a more efficient $(N, N^{1/2})$ sorting algorithm which requires only $O(\log \log N)$ rounds [25]. A still more efficient (N, S) sorting algorithm, named Columnsort, was created by Leighton [18]. Columnsort sorts N data items in four rounds, each of which consists of sorts of size $O(N^{2/3})$. Recursive calls to Columnsort yield an (N, S) sorting algorithm requiring $O((\log N/\log S)^{\beta})$ rounds, where $\beta = 2/(\log 3 - 1) \approx 3.419$, for any value of S where $2 \leq S \leq N$. In particular, if $S = N^{\varepsilon}$ for any positive constant ε, this recursive use of Columnsort requires only a constant number of rounds. In this paper, we will present an (N, S) sorting algorithm called Cubesort. In order to specify the performance of Cubesort, the following notation will be required.

The function $\log_2 X$ will be denoted $\log X$. The function $\log^{(0)} X \equiv X$ and for all integers $k > 0$, $\log^{(k)} X \equiv \log(\log^{(k-1)} X)$. The function $\log^* X \equiv k$ if k is the smallest nonnegative integer such that $\log^{(k)} X \leq 1$.

Given this notation, Cubesort is an (N, S) sorting algorithm which requires $O((25)^{(\log^* N - \log^* S)}(\log N/\log S)^2)$ rounds for any value of S, where $2 \leq S \leq N$. In particular, if $S \geq \log^{(k)} N$ for some constant k, then Cubesort requires $O((\log N/\log S)^2)$ rounds. Thus, Cubesort requires fewer rounds than Columnsort when S is small with respect to N.

Cubesort can also be used to obtain an efficient sorting algorithm for several distributed memory parallel computers. Let N be the number of data items to be sorted, let P be the number of processors available (it is assumed that $P \leq N$), and let $S = N/P$. The N data items are initially stored S per processor in a distributed memory parallel computer. The problem is to rearrange the data items so that they are in sorted order. No assumptions will be made about the range of values to be sorted, so only comparison-based sorts will be used. It will be assumed that the data items are distinct, as it is easy to create distinct values by using the starting positions of the data items to break ties.

A large number of different topologies have been proposed for distributed memory parallel computers. One important class consists of the hypercube [16], the shuffle-exchange [19, 26], and the cube-connected cycles [22] architectures. These three types of parallel computers will be called the CT (cube-type) computers.

One of the earliest parallel sorting algorithms is Batcher's bitonic sort [5], which sorts $N = P$ items on a CT computer in $O(\log^2 N)$ time [22, 26]. The bitonic sort is composed of a fixed sequence of *compare–exchange* operations. A compare–exchange operation consists of comparing two items and exchanging them conditionally, based on the result of the comparison.

In order to sort on a CT computer when $N > P$, a general technique devised by Baudet and Stevenson can be used [6]. Baudet and Stevenson's technique, which is actually an application of an observation made by Knuth [17], converts any compare–exchange sorting algorithm for the case $S = 1$ into a sorting algorithm for the case $S > 1$. The algorithm for the case $S > 1$ first sorts the S items per processor. Then the algorithm for the case $S = 1$ is implemented, except each compare–exchange operation is replaced by a *merge–split* operation. Each merge–split operation consists of merging two sorted lists of length S and splitting the resulting list at the median. By applying their technique to the bitonic sort, Baudet and Stevenson obtained an $O(S \log S + S \log^2 P)$ time sorting algorithm for shuffle–exchange computers. Their algorithm provides an optimal speed-up over sequential comparison sorting when $P = O(2^{\sqrt{\log N}})$. A similar algorithm was presented by Johnsson [16] for the hypercube computer. Johnsson's algorithm has the same asymptotic complexity as that of Baudet and Stevenson.

More recently, Aggarwal and Huang [1] created a parallel sorting algorithm based on the recursive application of Leighton's Columnsort [18]. For any S where $2 \leq S \leq N$, Aggarwal and Huang's algorithm runs on CT computers in $O(S \log N(\log N/\log S)^{\alpha})$ time, where $\alpha = (3 - \log 3)/(\log 3 - 1) \approx 2.419$. In contrast, Cubesort yields a sorting algorithm that runs in $O((25)^{(\log^* N - \log^* S)} S \log^2 N/\log S)$ time on CT computers [11, 12, 13, 14]. Thus, for extremely small values of S, Baudet and Stevenson's merge–split implementation of bitonic sort is faster than Cubesort. In particular, merge–split bitonic sort has better performance than Cubesort when $S < \log^{(k)} N$, where $k = \log^* N - \log^*(\log^* N) - \Theta(1)$, while Cubesort has better performance for larger values of S. Also, note that if $S \geq \log^{(k)} N$ for some constant k, Cubesort requires $O(S \log^2 N/\log S)$ rounds. Whenever $S = N^{\varepsilon}$ for any positive constant ε, both Aggarwal and Huang's algorithm and Cubesort are optimal with respect to sequential comparison sorting. However, as S becomes smaller with respect to N, Cubesort outperforms Aggarwal and Huang's algorithm.

A number of results for related problems have been obtained by other researchers. Reif and Valiant created a randomized algorithm that sorts $N = P$ items on a cube-connected cycles computer in $O(\log N)$ time with overwhelming probability [23]. Following the original appearance of

Cubesort, Plaxton deleloped an algorithm for sorting on hypercube computers that he called Smoothsort [21]. Let $X = \log^{1.5} P \log\log P$. When $S \leq X$ Smoothsort requires $O(\log S \log^3 P)$ time, and when $S > X$ Smoothsort requires $O(S \log^{1.5} P \log^{-0.5}(S/X) + \log S \log^3 P)$ time. The hypercube models used by Cubesort and Smoothsort are somewhat different. In particular, Smoothsort uses different dimensions of the hypercube simultaneously, while Cubesort uses only one dimension at a time. Smoothsort can also be implemented on shuffle–exchange computers, in which case it runs in $O((N/P)\log^2 P/\log(N/(P \log P)))$ time. A more recent algorithm developed by Plaxton, called Squaresort, runs in $O((N/P)\log^2 P/\log(N/P))$ time on a shuffle–exchange computer [20]. A still more recent algorithm developed by Cypher and Plaxton, called Sharesort, runs on CT computers in $O(\log N \log\log^2 N)$ time when $N = P$ [10]. Finally, optimal parallel sorting algorithms have been developed for models of parallel computation other than the CT computer [3, 8].

The remainder of this paper is arranged as follows. The basic Cubesort algorithm is presented in Section 2 and it is proven correct in Section 3. The basic Cubesort algorithm is then extended and analyzed in Section 4. The implementation of Cubesort on CT computers is described in Section 5 and conclusions are given in Section 6.

2. Cubesort

This section presents the basic Cubesort algorithm. This basic algorithm will operate under the restriction that $S \geq \log^4 N$. Extensions to this basic algorithm which relax this restriction will be given in Section 4.

In order to present Cubesort, two new dependent parameters, M and D, will be needed. Let $M^2 = S$, let $m = \log M$ and let $D = \log N/\log M$. In order to simplify the discussion it will be assumed that m and D are integers and that $D \geq 3$. Note that because $S \geq \log^4 N$, it follows that $M \geq \log^2 N$ and $D < \log N$, so $M \geq (D-1)(D-2)$. This relationship between M and D will be necessary for proving that Cubesort works correctly.

Cubesort sorts the contents of an array A containing N items by repeatedly partitioning the array into small groups and sorting within the groups. A total of D different partitions, denoted $R_1, R_2, \ldots, R_D$ will be used. For any i where $1 \leq i \leq D$, any two array locations K and L are in the same group in partition R_i if the binary representations of K and L differ only in bit positions $\max((i-2)m, 0)$ through $im - 1$ (assuming the least significant bit is in position 0). Note that R_1 partitions the array into N/M groups of size M, and for all i where $2 \leq i \leq D$, R_i partitions the array into N/M^2 groups of size M^2.

The following definitions and notation will be used to describe Cubesort. If X is a nonnegative integer then the y-bit binary representation of X will be written as $X = (X_{(y-1)}, X_{(y-2)} \ldots X_{(0)})$ and the ith bit of X will be denoted by $X_{(i)}$ (again, the 0th bit is the least significant). A *j-cube*, where $0 \leq j \leq D$, is a set of M^j locations in the array A that differ in only the jm least significant bits. That is, array locations K and L are in the same j-cube if $K_{(i)} = L_{(i)}$ for all $i \geq jm$. Note that the entire array A forms a single D-cube. Each j-cube, where $0 \leq j \leq D$, is classified as

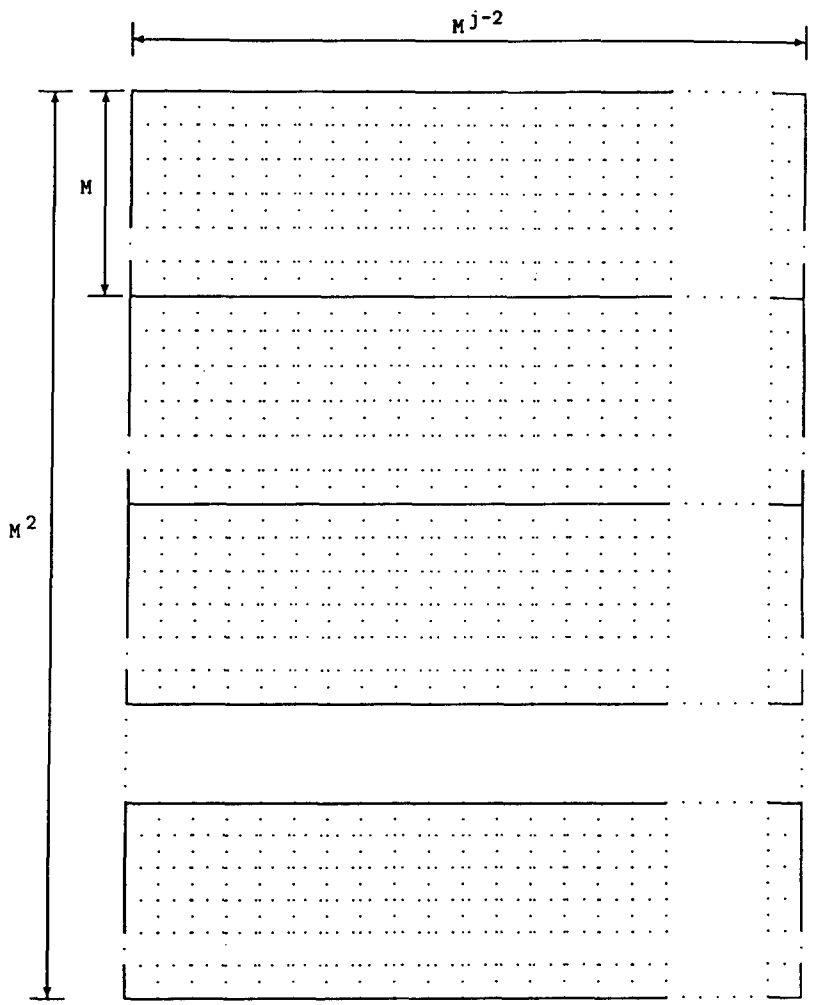


FIG. 1. A j-array.

being either *even* or *odd*. Note that for any two locations K and L in a j-cube, where $0 \leq j \leq D$, $K_{(jm)} = L_{(jm)}$. A j-cube, where $0 \leq j < D$, is even if it contains locations L such that $L_{(jm)} = 0$ and it is odd otherwise. The D-cube that consists of the entire array A is defined to be even.

It is often helpful to view a j-cube in the form of a two-dimensional array. A *j-array*, where $2 \leq j \leq D$, is an $M^2 \times M^{j-2}$ array of the items in a j-cube, where the items are placed in the array in row major order (see

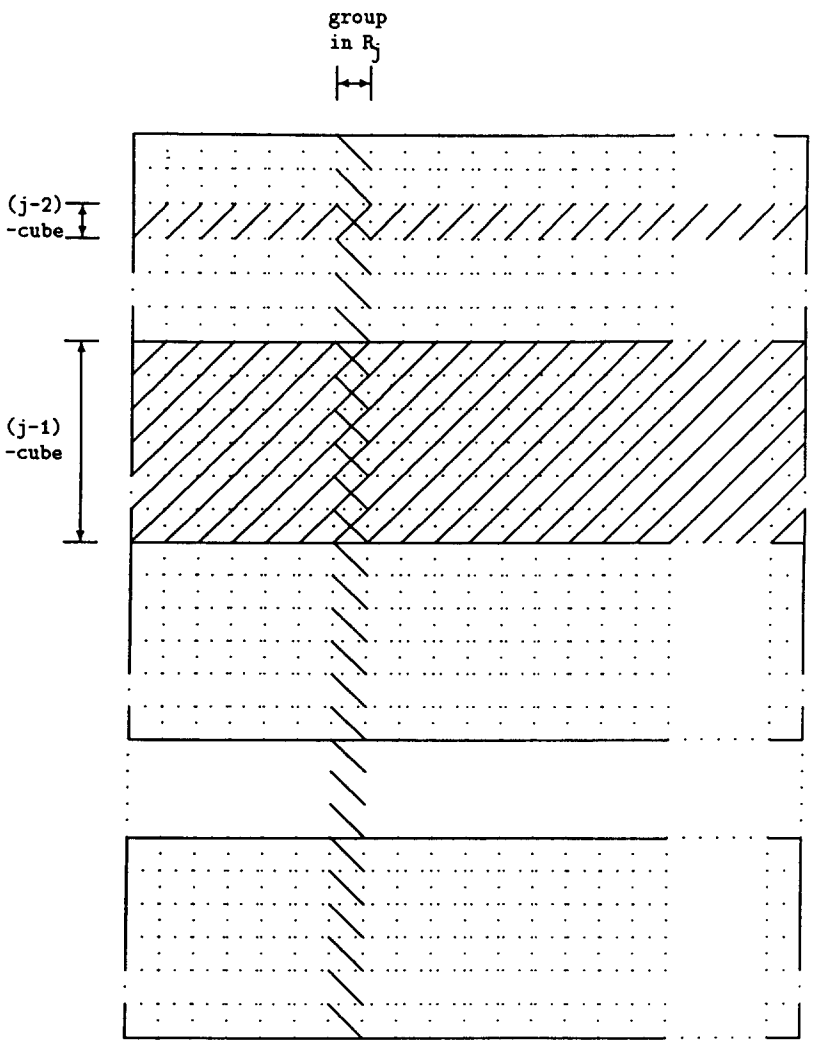


FIG. 2. Divisions of a j-array.

Fig. 1). Thus each $(j-2)$-cube forms a row in a j-array and each $(j-1)$-cube forms a band of M consecutive rows in a j-array. Also, each column in a j-array is a group in partition R_j (see Fig. 2).

Cubesort uses only two subroutines, *Sort_Ascending* and *Sort_Mixed*. The subroutine Sort_Ascending(i) sorts the groups in partition R_i in ascending order. The subroutine Sort_Mixed(i) sorts the groups in partition R_i that are in even i-cubes in ascending order, while it sorts the groups in R_i that are in odd i-cubes in descending order.

The basic Cubesort algorithm consists of three phases. A description of the algorithm is given next.

Phase 1. Perform the following operations:

- For $i = 2, 3, \ldots, D$ call Sort_Ascending(i).
- For $i = 2, 3, \ldots, D$ call Sort_Ascending(i).

Phase 2. Perform the following recursive call:

- If $D = 3$ call Sort_Mixed(2).
- If $D > 3$ call Cubesort recursively to sort the even $(D-1)$-cubes in ascending order and the odd $(D-1)$-cubes in descending order.

Phase 3. Perform the following operations:

- For $i = D, D-2, D-4, \ldots, k$, where $k = 2$ if D is even and $k = 1$ if D is odd, call Sort_Ascending(i).

3. Proof of Correctness

It will now be shown that the basic Cubesort algorithm defined in the previous section does indeed sort the array A when $S \geq \log^4 N$. The proof of correctness uses the *zero–one principle*, which states that "if a network with n input lines sorts all 2^n sequences of 0's and 1's into nondecreasing order, it will sort any arbitrary sequence of n numbers into nondecreasing order" [17]. Although the zero–one principle is stated in terms of sorting networks, it applies to any sorting algorithm that is composed of a fixed sequence of compare–exchange operations. In order to make the zero–one principle apply to Cubesort, the routines *Sort_Ascending* and *Sort_Mixed* should be implemented by using a sorting technique (such as Batcher's bitonic sort) that consists of a fixed sequence of compare–exchange operations. Of course, in practice any sorting algorithm could be used to implement the *Sort_Ascending* and *Sort_Mixed* routines without affecting the correctness of the algorithm.

In keeping with the zero–one principle, the following discussion will assume that the input consists entirely of 0's and 1's. Before proving the correctness of Cubesort, some additional terms will be needed. A set of items is *clean* if it contains only 0's or if it contains only 1's, and it is *dirty* otherwise (thus a dirty set contains at least one 0 and at least one 1). A sequence of 0's and 1's is *ascending* if it has the form 0^x1^y, where x and y are nonnegative, and it is *descending* if it has the form 1^x0^y, where x and y are nonnegative. A sequence is *monotonic* if it is either ascending or descending. A sequence of 0's and 1's is *bitonic* if it has the form $0^x1^y0^z$ or the form $1^x0^y1^z$, where x, y, and z are nonnegative. A j-array is *cross-sorted* if all of its rows are monotonic and if it has at most one ascending dirty row and at most one descending dirty row. A j-array is *semi-sorted* if all of its rows are bitonic and if it has at most one dirty row. A j-array is *block-sorted in ascending (descending) order* if it consists of x rows containing only 0's (1's), followed by y dirty rows, followed by z rows containing only 1's (0's), where x, y, and z are nonnegative. A j-cube is cross-sorted (or semi-sorted or block-sorted) if its corresponding j-array is cross-sorted (or semi-sorted or block-sorted).

It will be shown that after Phase 1 there are at most two dirty $(D-1)$-cubes in the entire D-cube. Furthermore, if there are two such dirty $(D-1)$-cubes then they must be adjacent (that is, together they form a set of $2M^{D-1}$ consecutive array locations).

Phase 2 then sorts the even $(D-1)$-cubes in ascending order and the odd $(D-1)$-cubes in descending order. After Phase 1, if there are two dirty $(D-1)$-cubes in the D-cube, then they must be adjacent within the D-array, so one of them must be even and the other one must be odd. Thus the at most two dirty $(D-1)$-cubes in the D-cube are sorted in opposite directions during Phase 2. It will be shown that this implies that the D-cube is cross-sorted after Phase 2. Phase 3 then completes the sort by merging together these sorted $(D-1)$-cubes. The merging procedure used in Phase 3 is similar to the bitonic merge algorithm. The proof of correctness of Cubesort follows.

LEMMA 1. *If a j-array originally has v dirty rows and if the columns of the j-array are then sorted in ascending (descending) order, the resulting j-array will be block-sorted in ascending (descending) order and will contain no more than v dirty rows.*

Proof. This lemma follows immediately from considering the numbers of 0's and 1's in each column. □

LEMMA 2. *After calling* Sort_Ascending(i) *for $i = 2, 3, \ldots, j$, there are at most $j-1$ dirty $(j-1)$-cubes in each j-cube.*

Proof. Proof is by induction on j. The basis is $j = 2$. In this case, the claim is that there is at most one dirty 1-cube in each 2-cube after calling Sort_Ascending(2). Sort_Ascending(2) sorts each group in partition R_2 in ascending order. Because each group in R_2 is a 2-cube, and because a sorted 2-cube can contain at most one dirty 1-cube, the basis is proven.

The induction hypothesis (I.H.) is that the lemma is true for all j, $2 \leq j < k$, for some $k > 2$. It will be shown that the I.H. implies that the lemma is true for $j = k$. From the I.H., it follows that after the calls to Sort_Ascending(i) for $i = 2, 3, \ldots, k - 1$ there are at most $k - 2$ dirty $(k - 2)$-cubes in each $(k - 1)$-cube (see Fig. 3). Because there are M

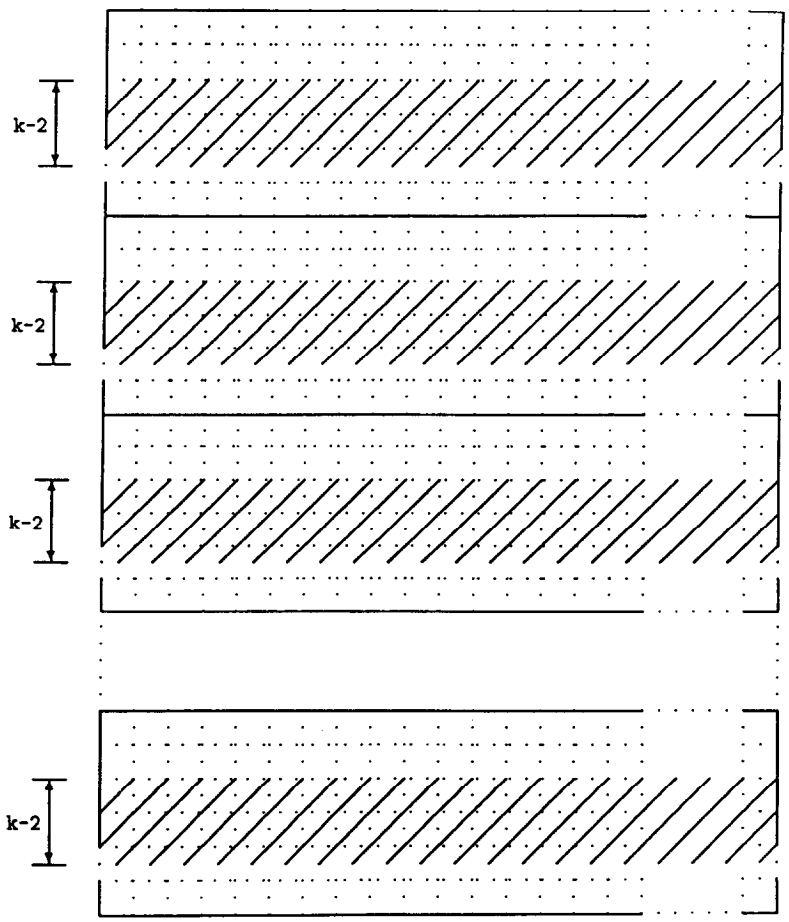


FIG. 3. A k-array with $k - 2$ dirty rows per $(k - 1)$-cube. Dirty rows are marked with slashes.

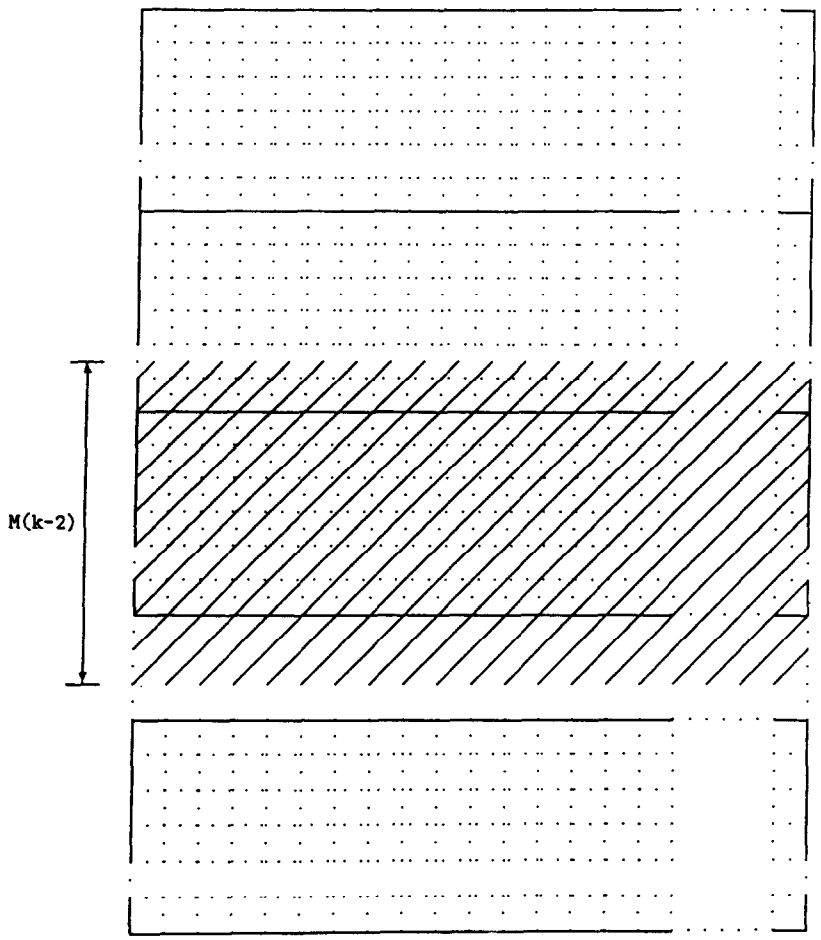


FIG. 4. A k-array with $M(k-2)$ dirty rows.

$(k-1)$-cubes in each k-cube, there are at most $M(k-2)$ dirty $(k-2)$-cubes in each k-cube. That is, each k-array has at most $M(k-2)$ dirty rows. Then the call to Sort_Ascending(k) sorts the columns of each k-array in ascending order. From Lemma 1, it follows that after calling Sort_Ascending(k) there will be at most $M(k-2)$ dirty rows in each k-array, and these dirty rows will be consecutive (see Fig. 4). Because each $(k-1)$-cube consists of M consecutive rows, there are at most $k-1$ dirty $(k-1)$-cubes in each k-cube. □

LEMMA 3. *After Phase* 1 *there are at most two dirty* $(D-1)$*-cubes in the* D*-cube and these dirty cubes are adjacent to one another.*

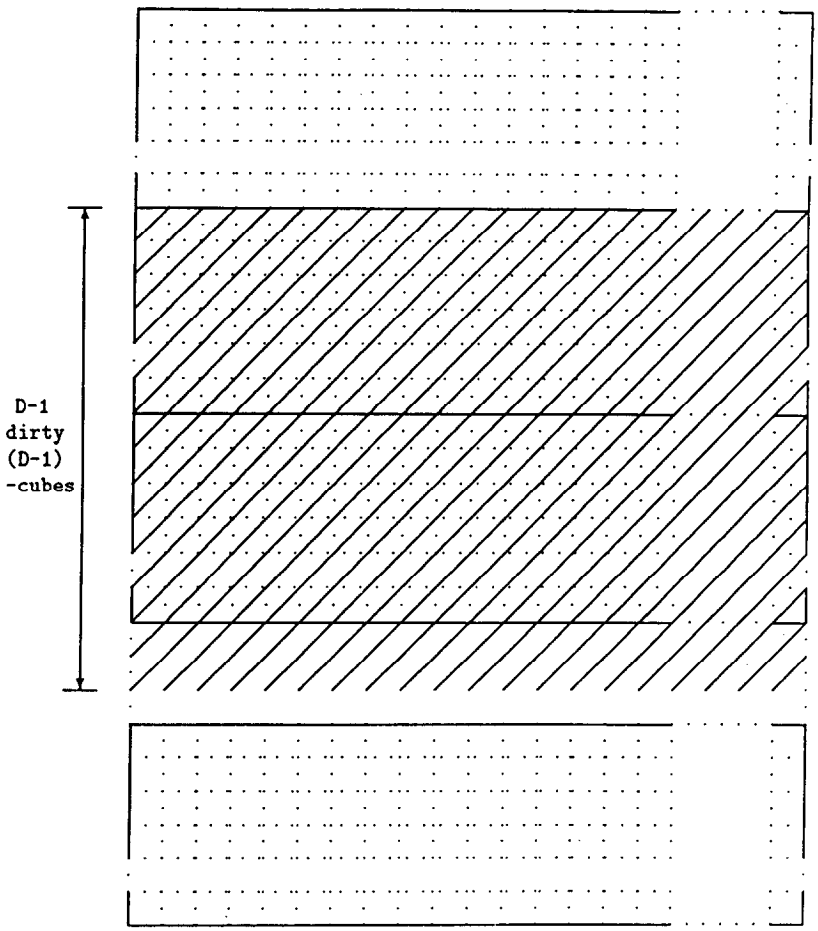


FIG. 5. A D-cube with $D - 1$ dirty $(D - 1)$-cubes.

Proof. It follows from Lemma 2 that after calling Sort_Ascending(i) for $i = 2, 3, \ldots, D$, there are at most $D - 1$ dirty $(D - 1)$-cubes in the D-cube (see Fig. 5). Next consider the effect of the following calls to Sort_Ascending(i) for $i = 2, 3, \ldots, D - 1$. Each of these calls sorts groups that are completely contained in a single $(D - 1)$-cube, so these sorts do not make any clean $(D - 1)$-cubes dirty. Also, it follows from Lemma 2 that after these calls each $(D - 1)$-cube contains at most $D - 2$ dirty $(D - 2)$-cubes (see Fig. 6). Now consider the effect of the final call to Sort_Ascending(D) in Phase 1. This call sorts the columns of the D-array in ascending order. Lemma 1 guarantees that after this final sort, each

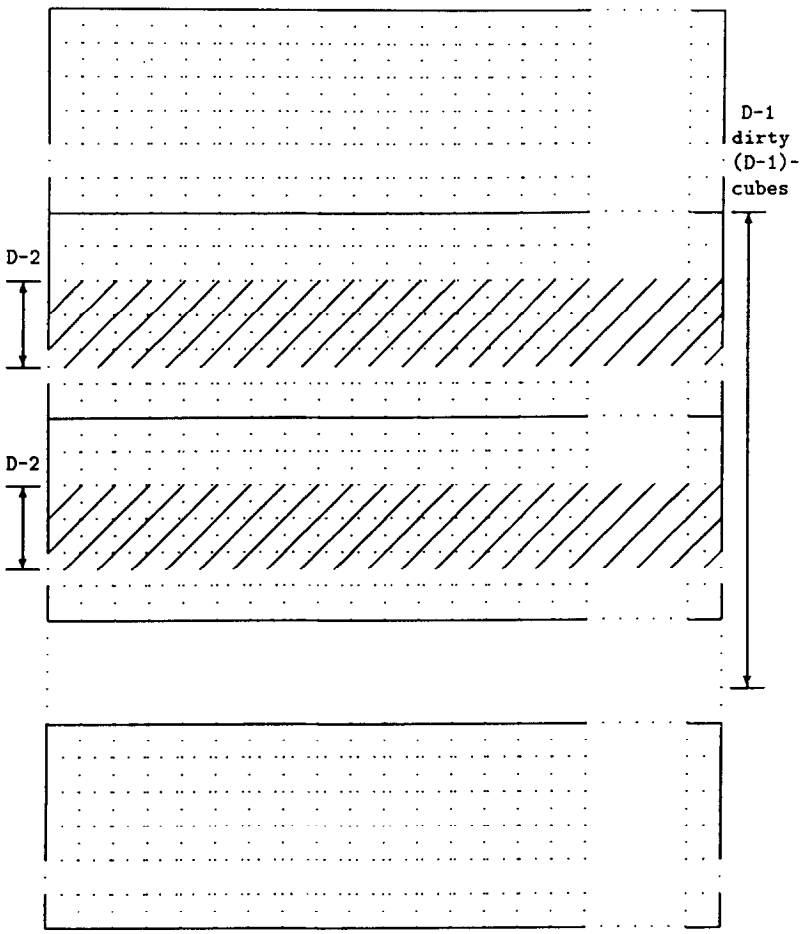


FIG. 6. $D - 2$ dirty rows per dirty $(D - 1)$-cube.

D-array will be block-sorted in ascending order and will have at most $(D - 1)(D - 2)$ dirty rows (see Fig. 7). But recall that $(D - 1)(D - 2) \leq M$ and that each $(D - 1)$-cube consists of M consecutive rows. Therefore, after Phase 1 there are at most two dirty $(D - 1)$-cubes in the D cube, and if there are two such dirty cubes then they must be adjacent. □

LEMMA 4. *If a j-array is originally semi-sorted or cross-sorted and if the columns of the j-array are then sorted in ascending (descending) order, the resulting j-array will be semi-sorted and it will be block-sorted in ascending (descending) order.*

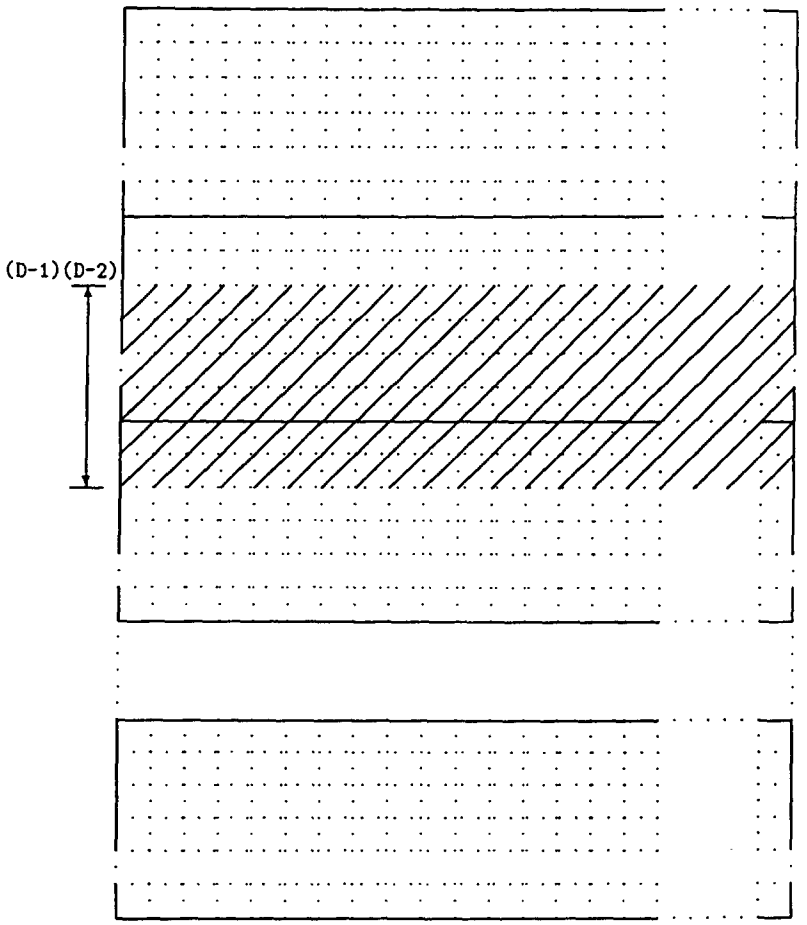


FIG. 7. $(D-1)(D-2)$ consecutive dirty rows.

Proof. Follows immediately from an examination of the possible cases. The possible interactions of the dirty rows in a cross-sorted array are illustrated in Fig. 8. □

LEMMA 5. *Let x be an integer where $1 \leq x \leq D$. If an x-cube is originally either semi-sorted or cross-sorted then after calling Sort_Ascending(i) for $i = x, x-2, x-4, \ldots, k$, where $k = 2$ if x is even and $k = 1$ if x is odd, the x-cube will be sorted in ascending order.*

Proof. The proof is by induction on x. The basis is when $x = 1$ or $x = 2$. In this case there is a single call to Sort_Ascending(x), which sorts

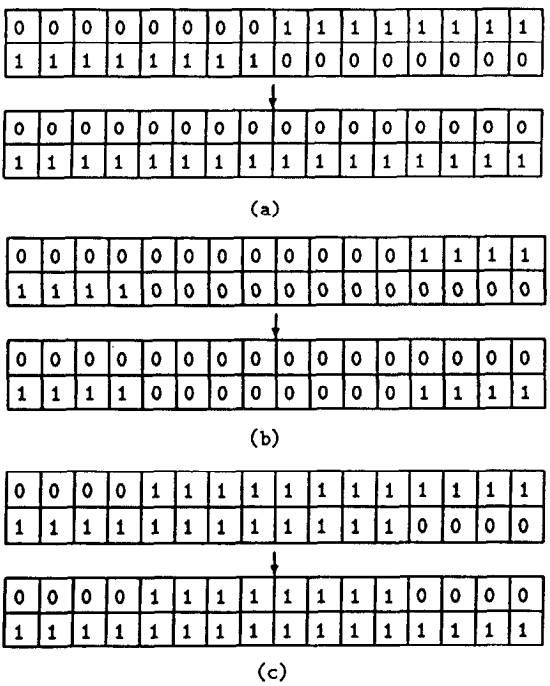


FIG. 8. Sorting the columns of a cross-sorted array. In each example the two upper rows yield the two lower rows.

the groups in partition R_x in ascending order. Because when $x = 1$ or $x = 2$ each group in R_x is an x-cube, the proof for the basis is complete.

The induction hypothesis (I.H.) is that the lemma holds for all values of x, $1 \leq x < y$, for some y, $3 \leq y \leq D$. It will be shown that the I.H. implies that the lemma holds for $x = y$. Consider the effect that the call to Sort_Ascending(y) has on a y-cube that is either semi-sorted or cross-sorted (see Fig. 9). The call to Sort_Ascending(y) sorts the columns of the y-array in ascending order. Lemma 4 guarantees that after the call to Sort_Ascending(y), the resulting y-array will be semi-sorted and it will be block-sorted in ascending order (see Fig. 10). Thus the resulting y-array has a set of rows of 0's, followed by at most one dirty row, followed by a set of rows of 1's. Also, if there is a dirty row, that row must be bitonic.

Now consider the effect of the remaining calls to Sort_Ascending(i). The only row of the y-array that will be changed by these calls is the at most one dirty row, which must be bitonic. Because this dirty row is bitonic, when viewed as a $(y - 2)$-cube it must either be cross-sorted (see Fig. 11) or semi-sorted (see Fig. 12). As a result, the I.H. guarantees that

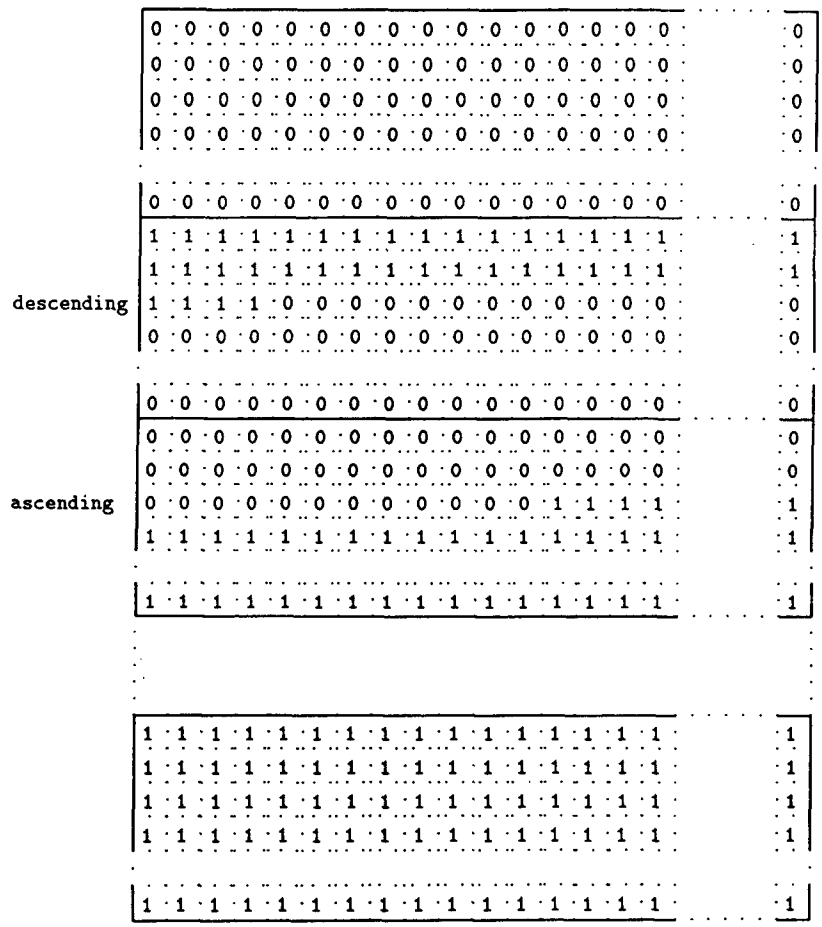


FIG. 9. A cross-sorted array.

the remaining calls to Sort_Ascending(i), where $i = y - 2, y - 4, \ldots, k$ with $k = 2$ if y is even and $k = 1$ if y is odd will sort this dirty row in ascending order. The y-cube will therefore be sorted in ascending order, which is the desired result. □

THEOREM 1. *The basic Cubesort algorithm sorts the D-cube into ascending order when* $S \geq n^4$ *and* $D \geq 3$.

Proof. The proof is by induction on D. The basis is when $D = 3$. In this case, it follows from Lemma 2 that after Phase 1 the entire 3-cube will

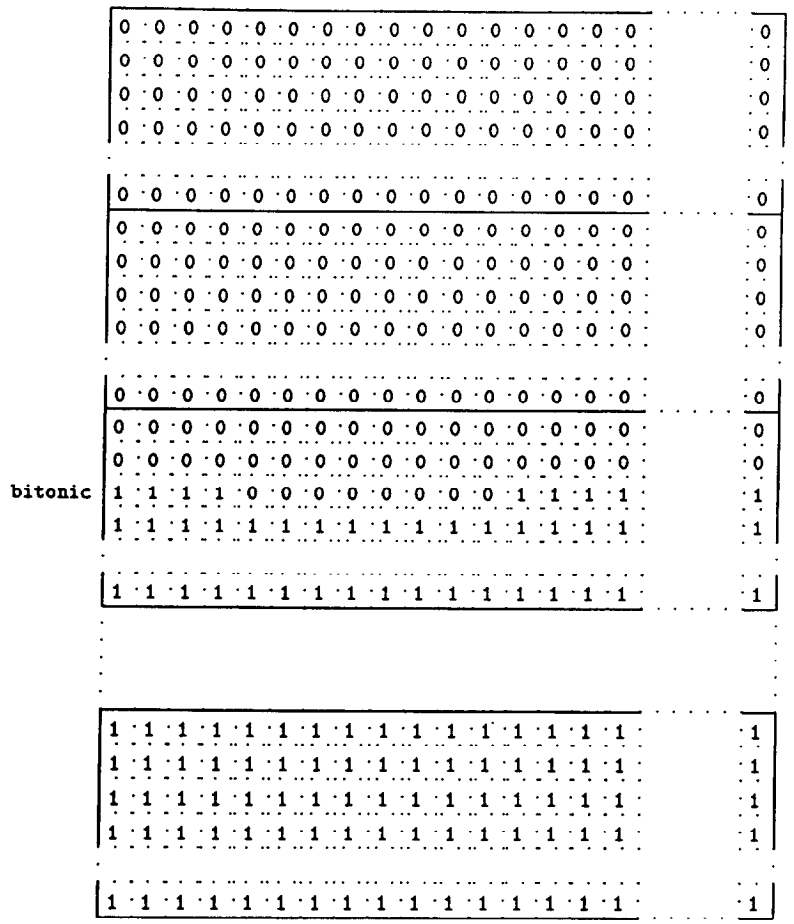


FIG. 10. A block-sorted and semi-sorted array.

contain at most two dirty 2-cubes, and that if there are two such dirty 2-cubes, they will be consecutive.

Phase 2 then consists of a call to Sort_Mixed(2), which sorts the groups in partition R_2 that are in even 2-cubes in ascending order and the groups in partition R_2 that are in odd 2-cubes in descending order. If after Phase 1 there were no dirty 2-cubes, then after Phase 2 there will still be no dirty 2-cubes, so the 3-array will have no dirty rows. If after Phase 1 there was one dirty 2-cube, then after Phase 2 that dirty 2-cube will be sorted, so the 3-array will have at most one dirty row which is sorted. If after Phase 1 there were two dirty 2-cubes, then after Phase 2 those dirty 2-cubes will be

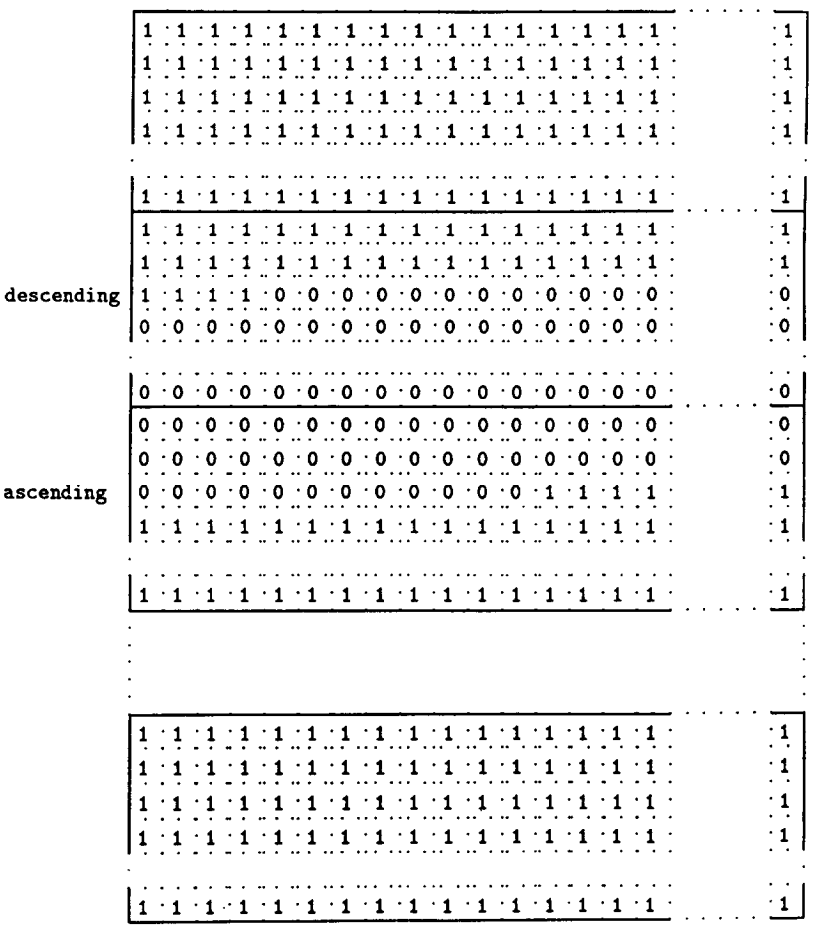


FIG. 11. A cross-sorted $(y - 2)$-array.

sorted in opposite directions, because they are adjacent and Sort_Mixed(2) sorts the even 2-cubes in ascending order and the odd 2-cubes in descending order. Note that a 2-cube that is sorted in ascending (descending) order can have at most one dirty 1-cube, and that the dirty 1-cube must be ascending (descending). Therefore, the 3-array will have at most two dirty rows which are sorted in opposite directions. Thus in any case, after Phase 2 the 3-cube will be cross-sorted. Finally, Lemma 5 guarantees that after Phase 3 the entire 3-cube will be sorted in ascending order.

The I.H. is that the theorem is true for all D, $D < y$, for some y, $4 \leq y$. It will be shown that the I.H. implies that the theorem is true for $D = y$.

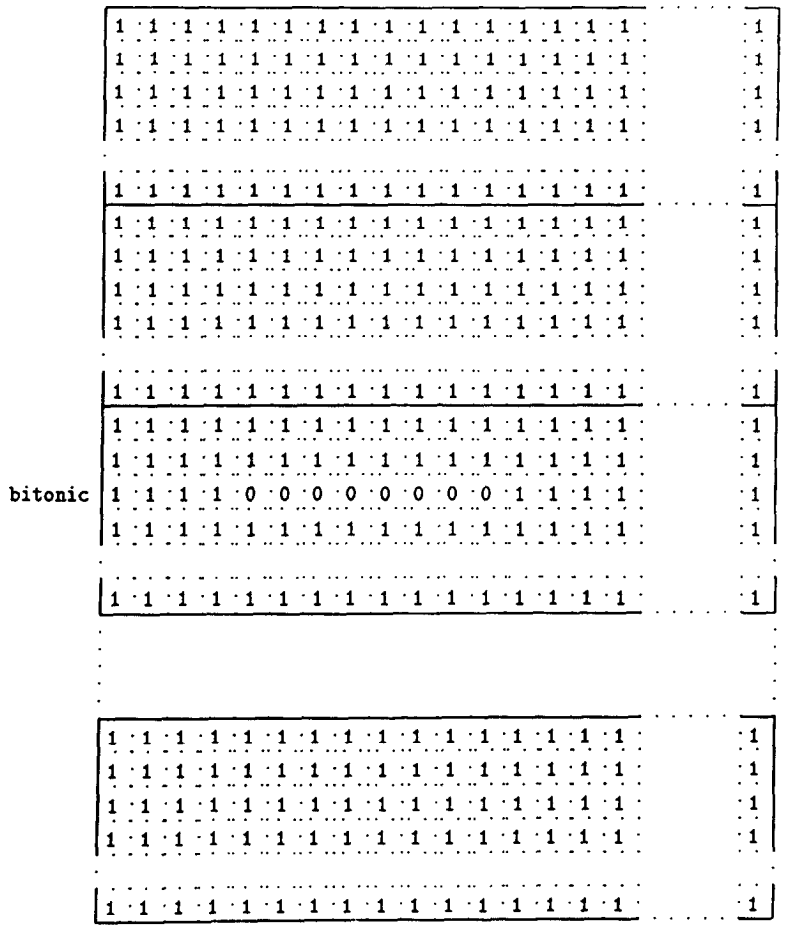


FIG. 12. A semi-sorted $(y - 2)$-array.

In this case, Lemma 3 guarantees that after Phase 1 of Cubesort, there are at most two dirty $(y - 1)$-cubes in the y-cube and that these dirty cubes must be adjacent.

Now consider the effect of Phase 2, which consists of recursive calls to Cubesort to sort the $(y - 1)$-cubes so that the even $(y - 1)$-cubes are sorted in ascending order and the odd $(y - 1)$-cubes are sorted in descending order. From the I.H., these calls do implement the desired sorts (see Fig. 9). Note that a $(y - 1)$-cube that is sorted in ascending (descending) order can have at most one dirty $(y - 2)$-cube, and that the dirty $(y - 2)$-cube must be ascending (descending). Therefore, after Phase 2

there are at most two dirty $(y-2)$-cubes in the y-cube, and these dirty $(y-2)$-cubes must be monotonic. Furthermore, because the at most two dirty $(y-1)$-cubes that are within a y-cube after Phase 1 must be adjacent, one of them must be even and the other one must be odd. Therefore, after Phase 2 each y-cube has at most one dirty $(y-2)$-cube that is ascending and at most one dirty $(y-2)$-cube that is descending. As a result, after Phase 2 every y-cube must be cross-sorted. It follows from Lemma 5 that after Phase 3 the entire y-cube will be sorted in ascending order. □

4. Analysis and Extensions

We will first analyze how many sorting rounds are required by the basic Cubesort algorithm. We will then analyze the performance of Cubesort when the basic algorithm is called in a recursive manner. For simplicity of exposition, we will often ignore the effects of floor and ceiling operators, but it can be shown that including their effects only increases the given bounds by a constant factor. Let $f(D)$ be the number of sorting rounds performed by the basic Cubesort algorithm for the given value of the parameter D. When $D = 3$, Phase 1 performs four rounds of sorting, Phase 2 performs one round of sorting, and Phase 3 performs two rounds of sorting, so $f(3) = 7$. When $D > 3$, Phase 1 performs $2D - 2$ rounds of sorting, Phase 2 performs $f(D-1)$ rounds of sorting, and Phase 3 performs $\lceil D/2 \rceil$ rounds of sorting, so when D is even $f(D) = (5/2)D - 2 + f(D-1)$ and when D is odd $f(D) = (5/2)D - 3/2 + f(D-1)$. The solution for this recurrence is $f(D) = \lceil (5D^2 - 2D - 12)/4 \rceil < (5/4)D^2$ for any $D \geq 3$. Because $D = \log N/\log M$ and $S = M^2$, $D = 2 \log N/\log S$. Therefore, the basic Cubesort algorithm is an (N, S) sorting algorithm which requires fewer than $(5/4)D^2 = 5(\log N/\log S)^2$ rounds.

We will now present extensions to the basic Cubesort algorithm for the case where $S < \log^4 N$. First, consider the case where $\log N \leq S < \log^4 N$. Recall that the basic Cubesort algorithm can be used as an $(N, \log^4 N)$ sorting algorithm which requires fewer than $5(\log N/\log(\log^4 N))^2 = (5/16)(\log N/\log\log N)^2$ rounds. Each of the sorts of $\log^4 N$ data items can then be implemented recursively with the basic Cubesort algorithm used as a $(\log^4 N, S)$ sorting algorithm. Each of these calls to Cubesort requires fewer than $5(\log(\log^4 N)/\log S)^2 = 80(\log\log N/\log S)^2$ sorting rounds, so the resulting algorithm is an (N, S) sorting algorithm which requires fewer than $25(\log N/\log S)^2$ rounds. In particular, when $S = \log N$ the resulting algorithm requires fewer than $25(\log N/\log\log N)^2$ rounds. We will call this $(N, \log N)$ sorting algorithm Cubesort2.

Next, consider the case where $S < \log N$ and $S = \log^{(k)} N$ for some positive integer k. In this case, we can use Cubesort2 recursively to obtain an (N, S) sorting algorithm. The depth of the recursion will be k (assuming the original call is at depth 1), and the resulting algorithm will require $(25)^k(\log N/\log S)^2$ rounds. Because $k = \log^* N - \log^* S$, the number of rounds required can also be expressed as $(25)^{(\log^* N - \log^* S)}(\log N/\log S)^2$.

Finally, consider the general case where $S < \log N$ and let k be the largest integer such that $S \leq \log^{(k)} N$. In this case, we can first call Cubesort2 recursively to a depth of k. This results in an $(N, \log^{(k)} N)$ sorting algorithm with fewer than $(25)^k(\log N/\log^{(k+1)} N)^2$ rounds. Then each of the sorts of size $\log^{(k)} N$ can be implemented with fewer than $25(\log^{(k+1)} N/\log S)^2$ rounds, so the resulting (N, S) sorting algorithm requires fewer than $(25)^{k+1}(\log N/\log S)^2$ rounds. These results are summarized in the following theorem.

THEOREM 2. *For any integers S and N where $2 \leq S \leq N$, Cubesort yields an (N, S) sorting algorithm which requires $O((25)^{(\log^* N - \log^* S)}(\log N/\log S)^2)$ rounds.*

5. IMPLEMENTATION ON CT COMPUTERS

This section examines the implementation of Cubesort on CT computers. Cubesort will be used to sort N data items on a CT computer with P processors where $P < N$. Define the parameter S to equal N/P. Our approach will be to use Cubesort to divide the problem of sorting N data items into a sequence of rounds, each of which performs P sorts of size S. Each round is implemented by first permuting the data so that each group of size S is local to a single processor. The round is then completed by sorting the S items per processor with a sequential algorithm.

The amount of time required by Cubesort on a CT computer can be divided into the time spent performing the local sorts and the time spent permuting the data between processors. As was shown in Theorem 2, a total of $O((25)^{(\log^* N - \log^* S)}(\log N/\log S)^2)$ rounds are required. Each sort of size S can be performed sequentially on a single processor in $O(S \log S)$ time. As a result, a total of $O((25)^{(\log^* N - \log^* S)} S \log^2 N/\log S)$ time is required to perform the local sorts. In the remainder of this section, we will show that this same bound applies to the amount of time required to permute the data between processors.

Our analysis of the time required to permute the data depends on some known results for a class of parallel algorithms called *Bit-Block* algorithms. The Bit-Block algorithms are a generalization of the Ascend and Descend algorithms defined by Preparata and Vuillemin [22]. A Bit-Block

algorithm operates on an array A that contains N items, numbered $0, 1, \ldots, N-1$. In each stage of a Bit-Block algorithm the array A is partitioned into groups and all calculations are required to occur entirely within these groups. The parameter B, called the *block size* of the algorithm, determines the size of the groups into which the array is partitioned. It is required that $B = 2^b$, where b is an integer.

Let $n = \log N$. A Bit-Block algorithm with a block size of B can use $n - b + 1$ different partitions. For each x, $0 \le x \le n - b$, partition number x, written Part(x, b, n), divides the array into groups of array locations that differ only in bit positions $b + x - 1, b + x - 2, \ldots, x$. For example, each group in the partition Part$(0, b, n)$ contains a set of B array locations that differ only in the b least significant bits. A Bit-Block algorithm can have many stages, each of which can utilize a different partition. However, all Bit-Block algorithms must start and end with the 0th partition. That is, if a Bit-Block algorithm uses the sequence of partitions Part(x_1, b, n), Part$(x_2, b, n), \ldots,$ Part(x_k, b, n) then x_1 and x_k must both be 0.

Bit-Block algorithms are important because they can be implemented efficiently on CT computers. The running time of a Bit-Block algorithm on a CT computer is related to the sequence of partitions that are used by the algorithm. Let $X = x_1, x_2, \ldots, x_k$ be the sequence of partitions used by a Bit-Block algorithm with a block size of B. Let the *bit distance of the algorithm*, written BitDist(X), be defined to be $k + \sum_{i=1}^{k-1} |x_{i+1} - x_i|$. The following theorem on the amount of time spent communicating data when implementing a Bit-Block algorithm is a combination of results by Cypher [9] and by Fishburn and Finkel [15].

THEOREM 3. *The communication required by a Bit-Block algorithm with a block size of B can be accomplished on a CT computer with N/B processors in $O(B \cdot \text{BitDist}(X))$ time, where $X = x_1, x_2, \ldots, x_k$ is the sequence of partitions used by the algorithm.*

Recall that the basic Cubesort algorithm uses a set of D partitions, $R_1, R_2, \ldots, R_D$ (see Section 2). Note that for $2 \le i \le D$, partition R_i is equivalent to partition Part$((i-2)m, \log S, \log N)$. Also, each group in partition R_1 is contained in a group of partition Part$(0, \log S, \log N)$. As a result, the basic Cubesort algorithm is a Bit-Block algorithm. Let $f(D)$ be the number of sorting rounds performed by the basic Cubesort algorithm for the given value of the parameter D. Also, let $g(D) = \text{BitDist}(X) - f(D)$, where X is the sequence of partitions used by the basic Cubesort algorithm with the given value of the parameter D. When $D = 3$, $g(3) = 6m$ (recall that $m = \log M = \log S^{1/2}$). When $D > 3$, $g(D) = 6Dm - 12m + g(D-1)$. The solution for this recurrence is $g(D) = 3D^2m - 9Dm + 6m$ for any $D \ge 3$. Therefore $g(D) = O(D^2m)$ and $\text{BitDist}(X) = O(D^2m + D^2) = O(D^2m)$. As a result, the communica-

tion required by the basic Cubesort algorithm can be implemented in $O(SD^2m) = O(S \log^2 N/\log S)$ time on a CT computer. Combining the time requirements for the local sorts and for the communication yields a total requirement of $O(S \log^2 N/\log S)$ time when the basic Cubesort algorithm is implemented on a CT computer.

The recursive use of Cubesort2 defined in Section 4 also yields a Bit-Block algorithm. First, consider the case where $S < \log N$ and $S = \log^{(k)} N$ for some positive integer k. In this case, Cubesort2 is used recursively to obtain an (N, S) sorting algorithm. Let X be the sequence of partitions used by the algorithm and let BitDist_i be the contribution to $\text{BitDist}(X)$ from a single call to Cubesort2 at a recursive depth of i (again the original call is at depth 1). For each value of i, $1 \le i \le k$, $\text{BitDist}_i = O((\log^{(i)} N)^2/\log^{(i+1)} N)$. For each value of i, $1 \le i \le k$, Cubesort2 is called at a recursive depth of i at most $(25)^{i-1}(\log N/\log^{(i)} N)^2$ times, so the total contribution to $\text{BitDist}(X)$ from all calls at recursive depth i is $O((25)^{i-1} \log^2 N/\log^{(i+1)} N)$ and $\text{BitDist}(X) = O((25)^k \log^2 N/\log S)$. Because $k = \log^* N - \log^* S$, the communication costs are $O((25)^{(\log^* N - \log^* S)} S \log^2 N/\log S)$ and the entire algorithm runs in $O((25)^{(\log^* N - \log^* S)} S \log^2 N/\log S)$ time on a CT computer. The analysis for the case in which S is not of the form $\log^{(k)} N$ for some integer k is analogous to that given in Section 4 and will not be repeated here. The results of this section are summarized in the following theorem.

THEOREM 4. *For any integers S and N where $2 \le S \le N$, Cubesort yields an $O((25)^{(\log^* N - \log^* S)} S \log^2 N/\log S)$ time algorithm for sorting N items on a CT computer with N/S processors.*

6. CONCLUSIONS

It has been shown that Cubesort is an efficient technique for partitioning a large sorting problem into a collection of smaller sorting problems, many of which can be solved in parallel. It has also been shown that Cubesort can be used to obtain an efficient sorting algorithm for CT computers when the number of data items exceeds the number of processors. Many algorithms rely on sorting as a key subroutine. For example, Nassimi and Sahni defined two data routing operations, called the RAR (random access read) and the RAW (random access write), and showed how they could be implemented by using sorting and a constant number of other routines, each of which is at least as fast as sorting [19]. Thus, Cubesort can be used to implement the RAR and RAW operations in the same time bounds as were given in Theorem 4. The RAR and RAW

operations can in turn can be used to efficiently simulate a Priority-CRCW PRAM with N processors and memory locations on a CT computer with P processors.

Finally, although this paper has concentrated on implementing Cubesort on CT computers, Cubesort can also be implemented efficiently on an EREW PRAM or a mesh connected computer of arbitrary (constant) dimension [13]. The implementation of Cubesort on an EREW PRAM has the same time complexity as the implementation on CT computers, and the implementation on mesh connected computers results in a running time proportional to the diameter of the computer.

Acknowledgments

This research was supported in part by an NSF Graduate Fellowship.

References

1. A. Aggarwal and M.- D. A. Huang, Network complexity of sorting and graph problems and simulating CRCW PRAMS by interconnection networks (preliminary version), *in* "Proceedings, 3rd Aegean Workshop on Computing, 1988," pp. 339–350.
2. A. Aggarwal and J. S. Vitter, The I/O complexity of sorting and related problems (extended abstract), *in* "Proceedings, 14th Intl. Colloquium on Automata, Languages and Programming, 1987," pp. 467–478.
3. M. Ajtai, J. Komlós, and E. Szemerédi, An $O(n \log n)$ sorting network, *Combinatorica* **3** (1983), 1–19.
4. M. J. Atallah, G. N. Frederickson, and S. R. Kosaraju, Sorting with efficient use of special-purpose sorters, *Inform. Process. Lett.* **27** (1988), 13–15.
5. K. E. Batcher, Sorting networks and their applications, *in* "Proceedings, AFIPS Spring Joint Computer Conf., 1968," pp. 307–314.
6. G. Baudet and D. Stevenson, Optimal sorting algorithms for parallel computers, *IEEE Trans. Comput.* **C-27**, No. 1 (1978), 84–87.
7. R. Beigel and J. Gill, Sorting n objects with a k-sorter, *IEEE Trans. Comput.* **39**, No. 5 (1990), 714–716.
8. R. Cole, Parallel merge sort, *SIAM J. Comput.* **17**, No. 4 (1988), 770–785.
9. R. Cypher, "Efficient Communication in Massively Parallel Computers," Ph.D. thesis, University of Washington, Dept. of Computer Science, August 1989.
10. R. Cypher and C. G. Plaxton, Deterministic sorting in nearly logarithmic time on the hypercube and related computers, *in* "Proceedings, 22nd Annual Symposium on Theory of Computing, 1990," 193–203.
11. R. Cypher and J. L. C. Sanz, "Algorithms for Reduced Hypercubes and Related Computers," Technical Report TR 89-04-05, University of Washington, Dept. of Computer Science, April 1989.
12. R. Cypher and J. L. C. Sanz, Cubesort: An optimal sorting algorithm for feasible parallel computers, *in* "Proceedings, 3rd Aegean Workshop on Computing, 1988," pp. 456–464.
13. R. Cypher and J. L. C. Sanz, "Optimal Sorting on Feasible Parallel Computers," Technical Report, IBM Almaden Research Center, November 1987.

14. R. CYPHER AND J. L. C. SANZ, Optimal sorting on reduced architectures, *in* "Proceedings, Intl. Conf. on Parallel Processing, 1988," Vol. III, pp. 308–311.
15. J. P. FISHBURN AND R. A. FINKEL, Quotient networks, *IEEE Trans. Computer*. **C-31**, No. 4 (1982), 288–295.
16. S. L. JOHNSSON, Combining parallel and sequential sorting on a boolean n-cube, *in* "Proceedings, Intl. Conf. on Parallel Processing, 1984," pp. 444–448.
17. D. E. KNUTH, "The Art of Computer Programming," Vol. 3, Addison–Wesley, Reading, MA, 1973.
18. F. T. LEIGHTON, Tight bounds on the complexity of parallel sorting, *IEEE Trans. Comput*. **C-34**, No. 4 (1985), 344–354.
19. D. NASSIMI AND S. SAHNI, Data broadcasting in SIMD computers, *IEEE Trans. Comput*. **C-30**, No. 2 (1981), 101–107.
20. C. G. PLAXTON, "Efficient Computation on Sparse Interconnection Networks," Ph.D. thesis, Stanford University, Computer Science Dept., September 1989.
21. C. G. PLAXTON, Load balancing, selection, and sorting on the hypercube, *in* "Proceedings, ACM Symposium on Parallel Algorithms and Architectures, 1989," pp. 64–73.
22. F. P. PREPARATA AND J. VUILLEMIN, The cube-connected cycles: A versatile network for parallel computation, *Comm. ACM* **24**, No. 5 (1981), 300–309.
23. J. H. REIF AND L. G. VALIANT, A logarithmic time sort for linear size networks, *J. Assoc. Comput. Mach*. **34**, No. 1 (1987), 60–76.
24. I. D. SCHERSON, S. SEN, AND A. SHAMIR, Shear sort: A true two-dimensional sorting technique for VLSI networks, *in* "Proceedings, Intl. Conf. on Parallel Processing, 1986," pp. 903–908.
25. C. P. SCHNORR AND A. SHAMIR, An optimal sorting algorithm for mesh connected computers, *in* "Proceedings, 18th Annual Symposium on Theory of Computing, 1986," pp. 255–263.
26. H. S. STONE, Parallel processing with the perfect shuffle, *IEEE Trans. Comput*. **C-20**, No. 2 (1971), 153–161.